From Abrupt to Gradual Drift in Federated Learning: Label-Free Reactive Retraining and a Service-Downtime Metric

Julio Amaral*, Hugo César†, Miguel Elias M. Campista†, Luiz Fernando Bittencourt*
*Institute of Computing, University of Campinas (UNICAMP), Brazil
j230537@dac.unicamp.br, bit@unicamp.br
†Federal University of Rio de Janeiro (UFRJ), Brazil
hugocesar@gta.ufrj.br, miguel@gta.ufrj.br

Abstract—Federated learning (FL) models deployed in production face concept drift: the data distribution shifts after deployment, and model quality degrades until a retraining action is taken; in real deployments the shift is often gradual, the fraction of corrupted inputs rising slowly over a transition interval whose duration is the drift width. We study reactive retraining in synchronous FL as the drift transition lengthens from abrupt to gradual. Each client monitors its unlabeled input stream with a local uncertainty-based detector (Monte Carlo dropout, Shannon entropy, and an ADWIN adaptive-window test); the server aggregates the local alarms over a short tick window and triggers at most once a single retraining round on each client's current mixture. The primary metric is the service downtime, the cumulative virtual time in which the served-model accuracy stays below a quality threshold. In 60 paired episodes (four calibrated corruptions, five seeds, three ramp durations of 50–200 s) over a fixed 600 s horizon, against a paired never-retraining baseline, detection succeeds in 59 of 60 episodes (14 of 15 in the Gaussian-noise scenario); the detection delay grows with the ramp duration for the three visual corruptions (98–140 s) and reaches 256 s for Gaussian noise; the reactive policy reduces the mean downtime from 408–562 s in the baseline to 0–178 s, reaching zero downtime at $\tau = 0.5$ in the longest-ramp fog scenario; and long ramps preserve clean-set accuracy for the visual corruptions (1–2 points loss instead of 9–21). Within the exercised regime (CIFAR-10, IID partition, ten clients, a single drift and at most one retraining round), a reactive label-free policy is effective against a never-retraining baseline; detection latency is scenario-dependent, Gaussian noise being the hardest case; a no-quorum variant detects about 14 s earlier with lower downtime, the measured price of the quorum's unquantified false-alarm protection; the collective trigger detects all staggered drift episodes with zero false alarms on drift-free streams.

Index Terms—federated learning, concept drift, drift detection, reactive retraining, unsupervised detection, service downtime.

I. Introduction

Federated learning (FL) trains a shared model from data that remains distributed across clients [1]. Once deployed, the model is served in an environment whose data distribution can change; such concept drift degrades the served model's quality until a retraining action is taken, and detecting it—usually without labels at inference time—is a prerequisite for triggering it. A common simplification in the literature is to inject drift abruptly: the distribution switches at a single instant. In practice, however, the shift is often gradual: new data appears mixed with old data, and the proportion of shifted inputs grows slowly over a transition interval whose duration is the drift width used in the drift-detection literature [3], [18]. We model this continuum as a linear ramp of the corrupted-data fraction from zero to one over a configurable number of monitoring ticks [3]. Our grid spans short ramps (the abrupt regime) and a long ramp in which the detector operates mid-transition.

Our reactive policy follows the architecture that the FL drift literature converged to: local detection, central decision [2], [4]–[10], [23], [24]. Each client runs an uncertainty drift detector (UDD) [11]: Monte Carlo (MC) dropout [12] over T stochastic forwards, mean Shannon entropy, and an ADWIN adaptive window test [13] per client. The server collects the per-client alarms over a sliding window of two monitoring ticks and triggers when the fraction of distinct flagged clients reaches a quorum. The triggered action is a single global retraining round on the mixture each client currently serves, in line with the explicit retraining budgets of DriftGuard [8] and ShiftEx [26] and Modyn's trigger/data-selection separation [20].

The primary metric is the service downtime: the cumulative virtual time in which the corrupted accuracy of the served model stays below a quality threshold τ. To our knowledge, no work in the reviewed FL drift literature quantifies the time a served model stays below a quality threshold; we detail this gap in Section II-E.

Our contributions are: (i) a service-downtime metric for FL under drift, computed as a stepwise integral over a fixed horizon; (ii) a collective quorum policy aggregating per-client label-free alarms over a short tick window, absent from the reviewed literature and closest to FL-MalDrift's participation gate [7], whose latency versus false-alarm-protection trade-off we characterize over quorum and window ablations and under staggered drift; (iii) a federated installation of UDD, with one detector per client sharing the global model, extending the centralized detector of Baier et al. [11]; (iv) an auditable experimental infrastructure pairing the reactive arm with

a no-retraining baseline from the same checkpoint, random state, stream, and horizon, validated before publication; and (v) an empirical characterization of gradual drift over 60 paired episodes.

II. Background and Related Work

A. Concept drift in federated learning

Concept drift is the change in the data distribution over time [19], [21]; the taxonomy distinguishes virtual drift (only $P(X)$ changes) from real drift (the conditional $P(Y \mid X)$ changes), and sudden, gradual, and recurrent drifts [19]. Staggered drifts, in which different clients drift at different times, are studied by Jothimurugesan et al. [4]. Gradual drift, the regime studied here, mixes the new concept with the old one with a growing proportion over time [2]; the transition duration is the drift width [3], [18].

Our scope is delimited to drifts accompanied by a change in $P(X)$ (covariate shift with preserved labels), the assumption under which label-free detection is structurally possible: label-free methods are deterministic functions of the inputs, so a change confined to $P(Y \mid X)$ produces no observable signal, a blind spot measured by Solozobov [17] and declared by Baier et al. [11].

B. Local detection, central decision

A recurring finding is that drift detection must be local. Jothimurugesan et al. [4] show that testing the aggregated error at the server fails during staggered transitions, because the errors of drifted and not-yet-drifted clients cancel out. FLARE [5], FLAME [6], FL-MalDrift [7], DriftGuard [8], FedDAA [9], FedPLC [10], FELK/FL-HVD [24], and Casado et al. [2] follow local detection with central decision: nine of the twelve concept-drift FL works we reviewed do so. The three exceptions: FLASH detects drift server-side from the magnitude of aggregated parameter updates [23], Stallmann et al. use a global fuzzy-clustering detector [25], and Adaptive-FedAVG forgoes detection altogether, adapting the server-side learning rate to raise the plasticity of the learning phase [27]. Our design follows the consensus; the specific signal, however, is label-free uncertainty (Section IV-A).

C. Label-free drift detection

The uncertainty drift detector (UDD) of Baier et al. [11] is the reference label-free method; Section IV-A describes our installation. ADWIN is chosen over DDM/EDDM because these require binomial inputs, whereas the entropy signal is not restricted to one [11]. Lobo et al. [14] connect concept drift and model uncertainty, and uncertainty estimators are known to degrade under dataset shift [15]. D3M [16] is a complementary label-free method based on model disagreement, with false-positive guarantees under the covariate-shift assumption; so is PUDD [28], a Pearson chi-square test on a prediction-uncertainty index whose significance level controls the false-positive rate, unlike UDD's mean-tracking ADWIN stage.

D. Retraining orchestration and budgets

DriftGuard [8] formalizes the retraining decision as a tuple $\pi^t = (\mathrm{Trig}, S, \theta)$: whether to retrain, which devices participate, and which parameters to update, reporting retraining-cost reductions of 80–83%. Modyn [20] separates trigger policies (time, volume, performance, drift) from data-selection policies; its drift trigger uses MMD (with KS among candidate statistics) and an adaptive percentile threshold. In the reviewed FL works, only DriftGuard [8] and ShiftEx [26] make the retraining budget explicit (a named quantity controlling the number of post-trigger rounds); we adopt a budget of one round on the distribution each client currently serves.

E. Metrics gap

The drift-detection benchmark of Cerqueira et al. [3] documents the lack of standardized temporal metrics (normalized detection delay, alarm rate, acceptance windows), as do FL drift surveys [19]. Detection latency [5], [11], retraining duration, and cost per retraining [8] are reported separately; none of the works we reviewed quantifies the time the served model stays below a quality threshold, the downtime metric we adopt, mirroring the SLO practice of MLOps. Downtime language exists in adjacent ML-systems work—federated unlearning measures service downtime as the unavailability incurred while retraining to erase data [29]—an availability notion (the service is offline), unlike time served online yet below a quality threshold.

III. Problem Statement

We consider synchronous FL with C clients and virtual time v, the abstract clock of our simulator. A model is trained on clean data for R rounds, then enters production at time v_0. Production data is served in monitoring ticks of Δt seconds, one unlabeled batch per client per tick.

Gradual drift model. A corruption applied to inputs X (labels preserved) is injected with a mixture fraction $f(v)$ that grows linearly from zero to one over N ticks:

$$f(v) = \min\left(1,\ \frac{v - v_0}{N\,\Delta t}\right), \quad v \geq v_0. \tag{1}$$

The ramp duration $N\Delta t$ is the drift width; its slope $1/N$ per tick is the independent variable. Here $N \in \{5, 10, 20\}$ (50, 100, 200 s at $\Delta t = 10$ s).

Quality threshold and downtime. Let $a(v)$ be the accuracy of the served model on a test set mixed at the current fraction $f(v)$. The quality threshold τ defines the service state: the model is unavailable when $a(v) < \tau$. The downtime over a fixed horizon $[v_0, v_0 + H]$ is

$$D \;=\; \sum_{i=1}^{m} (v_{i+1} - v_i) \cdot \mathbf{1}\big[a(v_i) < \tau\big], \tag{2}$$

where $v_1, \ldots, v_m$ are the evaluation instants and $v_{m+1} = v_0 + H$: the stepwise integral uses the last known quality, without interpolation, and includes the final interval up

to the horizon. The evaluation instants comprise tick boundaries, the retraining completion instant, and the horizon.

Paired evaluation. The reactive policy is compared with a baseline that observes the same stream from the same checkpoint and never retrains. Both arms share the same random state, production start, and horizon H; the baseline records the moment the policy would have triggered as a counterfactual.

IV. Proposed Method

A. Local uncertainty detection (UDD per client)

Each client c runs a local detector over the shared global model: (1) the model is set to evaluation mode with dropout layers re-enabled; (2) $T = 5$ stochastic forwards produce class probabilities; (3) the mean Shannon entropy $H = -\sum_k p_k \log p_k$ over the batch is computed; (4) the scalar feeds a per-client ADWIN with $\delta = 0.002$ [11], [13]. Our ADWIN implementation tests windows of at least 20 observations, because we feed one batch-mean entropy per tick per client rather than a per-instance stream. Each detector therefore receives 20 warm-up ticks on clean data before production starts, calibrating it without feeding the collective policy. The detector never mutates the model: training modes are restored after each update, and only the inputs X are observed.

B. Collective quorum policy

At each production tick, the server receives the per-client alarm flags and maintains a sliding window of the last two ticks. The flagged fraction is the number of distinct clients flagged in the window divided by the number of clients. When the fraction reaches the quorum $\theta = 0.30$ (3 of 10 clients), the server triggers, and a latch prevents any further decision in the episode. A single isolated alarm (a possible local false positive under continuous monitoring [18]) is not enough to trigger.

C. Reactive retraining on the current mixture

When the policy triggers at time t_d, each client's training dataset is replaced by a mixture with corrupted fraction $f(t_d)$, built with a deterministic per-client permutation, and a single synchronous FL round is executed with all clients. Each client holds 5,000 training images (an IID split of the 50,000-image training set) mixed at $f(t_d)$. Retraining learns the current distribution, not the future of the ramp. The budget is explicit (one round); if the required virtual time would exceed the remaining horizon, the decision is recorded and the action skipped. A retraining round consumes about 8 s of virtual time (one server timeout); in every detected episode the accuracy at the round's completion already meets τ, so the time-to-recovery equals the retraining duration.

TABLE I
Severity calibration from the clean checkpoint (strict interval $(0.25, 0.50)$).

Corruption	Severity	Accuracy
Gaussian noise	3	0.3783
Frosted-glass blur	4	0.4022
Motion blur	1	0.3493
Fog	4	0.4875

D. Evaluation on the current mixture

The accuracy series is measured on the full test set (10,000 images) mixed at the current fraction $f(v)$: at production start the test is clean (fraction zero), and it becomes fully corrupted only when the ramp completes; a fully corrupted test would overstate the degradation during the ramp. Mixing uses a deterministic permutation seeded per episode, so both arms are evaluated on the identical tensor at every instant.

E. Severity calibration

Each corruption is calibrated on a clean checkpoint: all severities 1–5 are evaluated on the corrupted test set, and the smallest severity whose accuracy falls strictly inside $(0.25, 0.50)$ is selected. The lower bound keeps the model well above chance (0.10 for ten classes), so recovery is achievable with a small budget; the upper bound guarantees the corruption pushes the service below τ.

V. Experimental Setup

A. Dataset and corruptions

We use CIFAR-10 with an IID partition across 10 clients and the full 10 classes in all phases. Four visual corruptions are implemented directly on tensors—Gaussian noise, frosted-glass blur, motion blur, and fog—with integer severities 1–5, per-seed generators, and preserved shapes and labels. They are inspired by, not a reproduction of, the CIFAR-10-C benchmark [22]; their internal parameters are project-specific: Gaussian noise adds perturbations with per-severity standard deviations of 0.08–0.24; frosted-glass blur permutes pixels within a 3×3 neighborhood and blends with a Gaussian blur (strength 0.2–1.0); motion blur convolves with a diagonal kernel of width 5–15; fog blends the image with a four-octave fractal noise map.

B. Calibration result

The calibration, executed once from a clean checkpoint, selected the severities in Table I (procedure in Section IV-E).

C. Grid

The study grid is the Cartesian product of the four calibrated corruptions, five seeds (42–46), and three ramp durations (5, 10, 20 ticks), totaling 60 paired episodes with a fixed production horizon of $H = 600$ s. Table II lists the fixed configuration.

TABLE II
Fixed configuration of every episode.

Parameter	Value
Dataset	CIFAR-10, IID, 10 clients
Initial rounds (clean)	20
Warm-up ticks (clean)	20
Tick duration	10 s
Batch size / epochs	32 / 1
Model	CNN 2.2M (4 conv, 2 fc)
Optimizer	Adam, lr 10^{-3}
Speed / timeout	uniform (p75; round $\approx$ 8 s)
Detector	ADWIN $\delta = 0.002$; MC dropout $T = 5$
Quorum / window	0.30 / 2 ticks
Retraining budget	1 round
Quality threshold τ	0.50
Production horizon	600 s
Seeds	42–46

TABLE III
Detection delay (s) and drift progress at the trigger (group means over 5 seeds; detection rate per group; the failed Gaussian seed at 100 s is excluded from its delay mean).

	Delay (s)			Drift progress	
Corruption	50 s	100 s	200 s	200 s	det.
Fog	100	112	138	0.69	5/5
Frosted-glass	100	108	136	0.68	5/5
Motion blur	98	112	140	0.70	5/5
Gaussian noise	164	158*	256	0.97	4/5*

Delay std across seeds: $\leq$6 s for the visual corruptions; 71–104 s for Gaussian noise. For the 50 and 100 s ramps the trigger occurs after the ramp ends, so the drift progress at the trigger is 1.00 by construction.

TABLE IV
Downtime (s): reactive arm versus paired no-retraining baseline (group means over 5 seeds; format: reactive / baseline).

	Ramp duration		
Corruption	50 s	100 s	200 s
Fog	58 / 550	24 / 504	0 / 408
Frosted-glass	68 / 560	40 / 524	3 / 454
Motion blur	68 / 562	50 / 530	14 / 466
Gaussian noise	130 / 558	178 / 526	112 / 448

Gaussian-noise dispersion is large (delay std up to 104 s; downtime std up to 184 s, range 38–540 s); visual corruptions have std $\leq$6 s (delay) and $\leq$10 s (downtime).

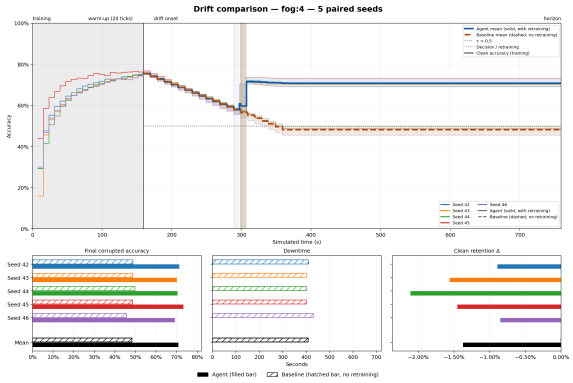

Fig. 1. Fog, 200 s ramp, five paired seeds. Top: corrupted accuracy over simulated time (solid: reactive, dashed: baseline; thick lines: means); $\tau = 0.5$ dashed; shaded bands: the retraining round. Bottom: final corrupted accuracy, downtime, and clean retention; the reactive arm never falls below τ (zero downtime).

VI. Results

We report group means over the five seeds and test two hypotheses: H1, the reactive policy reduces downtime relative to the no-retraining baseline; H2, the detection delay grows with the ramp duration. Primary measurements are the detection delay (production start to trigger), the drift progress at the trigger, and the downtime; secondary are clean retention (final minus pre-drift clean accuracy) and final corrupted accuracy.

A. Detection: 59 of 60 episodes

Detection succeeded in 59 of 60 episodes (14 of 15 for Gaussian noise); the only failure is Gaussian noise, 100 s ramp, seed 42. The delay grows with the ramp duration for the three visual corruptions, consistent with ADWIN's analysis of gradual change with constant slope [13]; Gaussian noise does not follow the trend.

B. Downtime: reactive policy dominates the baseline

The reactive policy reduces the mean downtime in all 12 groups, from 408–562 s to 0–178 s, with 59 wins in the 60 paired episodes. The fog scenario with the 200 s ramp reaches zero downtime at $\tau = 0.5$ (Fig. 1): the trigger occurs at a drift progress of 0.69 while the accuracy is still 0.568 (above τ), and the single round keeps it above τ (final corrupted accuracy 0.71).

Comparison with a first-alarm trigger. We also ran a first-alarm policy (quorum 0.10, one-tick window). Any retraining policy cuts mean downtime from 507.5 s (no retraining) to at most 62.2 s (87.7–91.7%), none worse than the baseline in any group; the quorum-0.30 arm, however, does not dominate the first-alarm trigger: the first-alarm reaches 42.1 s against 62.2 s for the reactive arm, improving downtime in all 12 groups; the quorum's cost is a mean detection-latency increase of about 13.8 s, examined in Section VI-C. The reactive-arm-versus-baseline contrast is ordinally robust to the quality threshold: across $\tau \in \{0.40, 0.45, 0.50, 0.55\}$ the reactive arm never exceeds its paired baseline in any of the 48 cells, though magnitudes are τ-specific (the mean contrast shrinks about 47% at $\tau = 0.40$; the reactive arm's worst case grows to 190 s at $\tau = 0.55$).

C. Collective-quorum analysis

All 10 clients raise an alarm in every detected episode (59 of 59): the local detectors are individually sensitive. The trigger fires at the quorum's minimum (0.30, three distinct clients in the two-tick window) in 21 of 59

episodes; Gaussian noise fires at the minimum in all detected episodes of the 100 s ramp. The long ramp spreads the alarms: the flagged fraction at the decision drops with the ramp duration (fog 0.88→0.42, frosted 0.90→0.56, motion 0.66→0.54).

Quorum and window sensitivity. The first-alarm trigger of Section VI-B (quorum 0.10, one-tick window) is the no-quorum extreme: it fires in 60 of 60 episodes and detects 2–50 s earlier than the default (121.3 s vs. 135.1 s mean delay), at the price of the quorum's protection against isolated false alarms (Section IV-B), not measurable here. The detection delay increases monotonically with the quorum but in small increments: raising it from 0.10 to 0.30 adds about 13.8 s of delay and about 20 s of downtime, and from 0.30 to 0.50 another 8.6 s and 8 s; the cost concentrates in Gaussian noise (up to +96 s at the 100 s ramp) and is at most 10 s elsewhere. The 0.50 quorum fires in 59 of 60 episodes, missing the same Gaussian-noise seed as the 0.30 default (at a different ramp): no quorum level introduces a new failure mode. A one-tick window at the default quorum is a wash overall (+3.7 s delay, −3.9 s downtime), swapping sign per group; the two-tick union shows no systematic benefit here.

Staggered drift. When clients drift at different ticks (client i drifts at production tick i, no ramp; 20 episodes), the collective trigger fires in 20 of 20 episodes at the quorum boundary (flagged fraction 0.30–0.40), corroborating alarms a one-tick window would miss. The policy avoids about 81% of the baseline downtime (600 s, permanent drift) on three of the four corruptions and 70% on Gaussian noise (116 s and 180 s of reactive-arm downtime, respectively); since the drift is instant and permanent, delay converts to downtime one-to-one (116 ≈ 108 + 8 s). The quorum's value is thus suggestive in the staggered regime; under simultaneous drift it is cheap insurance whose false-positive protection is not measurable here.

D. False-positive rate on drift-free streams

We ran 15 control episodes (five seeds times three ramp schedules) with no corruption: each client monitors a clean stream for the full 600 s horizon. The collective gate never fires—0 of 15 episodes record a retraining decision—and the local detectors raise zero alarms across the 9,000 client-tick evaluations. Eight isolated entropy peaks (scores above 1.0) occur, but ADWIN flags none: it tests for a distributional change, not a spike. The gate is thus silent on drift-free streams and sensitive under drift (59 of 60 episodes; measured false-positive rate 0.0%).

E. Clean retention and recovery behavior

First, long ramps preserve the clean-set accuracy (Table V): with the 200 s ramp the retraining set keeps roughly 30% of clean data and the clean-accuracy loss drops to 1–2 points (versus 9–21 on short ramps) for fog, frosted-glass, and motion blur; Gaussian noise does not benefit (trigger at drift progress 0.97). Second, the recovery is not temporary: no episode re-crosses τ within the horizon; the retrained model generalizes to the fully corrupted regime. In 9 of the 20 episodes with the 200 s ramp (all five fog runs, three frosted runs, one motion run) the accuracy never falls below τ. Group-mean final corrupted accuracy ranges from 0.60 to 0.71 for the reactive arm (excluding the failed episode at 0.33) and 0.35–0.48 for the baseline (Fig. 2).

TABLE V
Clean retention Δ (reactive arm; group mean ± std over 5 seeds).

Corruption	Ramp duration 50 s	100 s	200 s
Fog	-0.096 ± 0.012	-0.094 ± 0.014	-0.014 ± 0.005
Frosted-glass	-0.152 ± 0.008	-0.156 ± 0.014	-0.022 ± 0.005
Motion blur	-0.199 ± 0.017	-0.208 ± 0.020	-0.016 ± 0.002
Gaussian noise	-0.084 ± 0.013	-0.066 ± 0.034	-0.072 ± 0.016

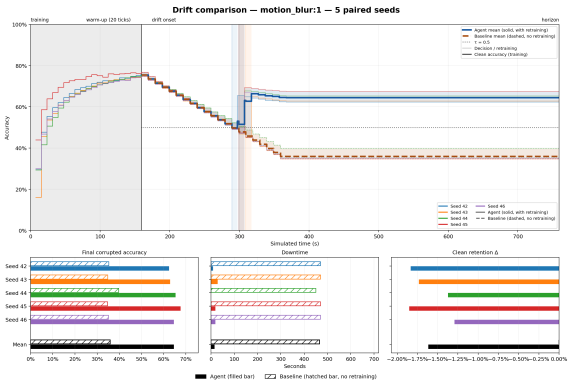

Fig. 2. Motion blur, 200 s ramp, five paired seeds. The reactive arm recovers within one round and stays above τ for most of the horizon; the baseline keeps degrading to about 0.35.

VII. Discussion

Monitoring is not free: over the virtual 80-tick episode the $T = 5$ detector costs about 6.8 TFLOPs (4,000 batch-32 forwards, 50 per tick across the ten clients) against about 8.0 TFLOPs for the single retraining round, a 0.85–0.96× ratio depending on the accounting, roughly 47% of the reactive compute. Across the ten clients the per-tick monitor occupies about 0.13 s, 1.3% of each 10 s tick; on the measured device (Apple M5, PyTorch 2.12.1) one detector update takes about 12.7 ms per client. Had we kept the published UDD setting $T = 50$, monitoring would exceed the retraining round by roughly 9×, which is why $T = 5$ was chosen as a cost decision.

The downtime conclusions are robust to a 10× longer round: recomputing downtime post hoc from the recorded series—charging the pre-retrain accuracy over $[t_d, t_d + d]$ and the first post-retrain record thereafter, with the controller's budget rule applied—an 80 s communication-dominated round still cuts the mean downtime from 507.5 s (baseline) to 122.2 s (76%); the per-group ranking, the zero-downtime fog case, and the detection-limited

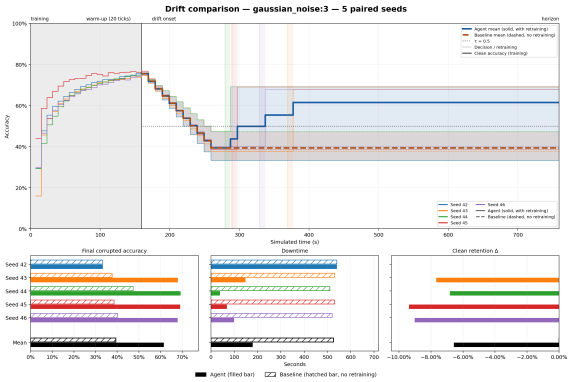

Fig. 3. Gaussian noise, 100 s ramp, five paired seeds. Seed 42 never triggers: its trajectory (solid blue) does not recover, while the other four seeds recover to about 0.68.

Gaussian behaviour are unchanged, the extra cost being exactly the pre-retrain interval the round extends. At $d = 800$ s the round no longer fits the 600 s production horizon and the budget rule would skip retraining in all 59 triggered episodes, degenerating the reactive policy to the baseline by policy rather than by training dynamics; the turning point is the smallest trigger-to-horizon slack, about 140 s in this grid.

A. Gaussian noise is the limiting scenario

Gaussian noise is the only scenario with incomplete detection. With the 100 s ramp and seed 42, the policy never triggers: only four of the ten clients raise an alarm in the whole episode, never three within the same two-tick window (maximum flagged fraction 0.20 at sparse ticks); the other four seeds detect within 120–210 s. As the window and the quorum are identical across seeds, the failure is best explained as a per-client detector miss (an interaction we cannot isolate). Without a decision the reactive arm degenerates exactly to the baseline (downtime 540 s). With the 200 s ramp the delays spread between 180 and 460 s (std 104 s), yet the reactive arm still cuts the baseline downtime by 75% (112 vs. 448 s), consistent with the risk ADWIN-style detectors carry under long, gentle drifts [18]. Gaussian noise is also the scenario with the highest reactive latency: the first-alarm trigger averages 77.3 s against 140.1 s for the quorum-0.30 arm—the seed-42 miss is the price of a trigger that depends on local detectability. Fig. 3 shows the failed seed.

B. Why the long ramp behaves better

For the visual corruptions the 200 s ramp has three favorable effects. (i) The trigger occurs mid-ramp, at a drift progress of 0.68–0.70, where the accuracy at the decision is 0.57 for fog (still above τ), 0.51 for frosted-glass, and 0.48 for motion blur. (ii) The retraining set mixes about 30% clean data, reducing catastrophic forgetting. (iii) The model retrained at a drift progress of $\approx$0.7 generalizes to the fully corrupted regime, so the recovery holds. Short ramps behave like the abrupt regime: detection completes after the ramp and the retraining set is almost fully corrupted. Gaussian noise triggers later (drift progress 0.97) and does not enjoy effects (i) and (ii).

C. Limitations

Our results are bounded by the following declared limitations. (i) Five seeds per group limit the precision of the dispersion estimate. (ii) The model is a small CNN of about 2.2 million parameters; absolute accuracies under corruption are modest, as expected for small architectures [22]. (iii) The scenario assumes a single drift, one retraining round, an IID partition, uniform speed, and a fixed horizon. (iv) The drift progress at the trigger is informative only for the 200 s ramp; on the 50 and 100 s ramps the trigger occurs after the ramp ends, so the value is 1.00 by construction. (v) The published UDD uses $T = 50$ forwards; our $T = 5$ reduces the monitoring budget as a cost trade-off, and the direct comparison between the two settings is not investigated here and remains future work; $\delta = 0.002$ is the fallback of the reference implementation rather than a calibrated value [11], [18]. (vi) Downtime is sampled at tick granularity plus the round-completion instant; the true crossing of τ can fall between ticks. (vii) Episodes ran on a CUDA GPU (PyTorch 2.5.1, CUDA 12.1) with non-deterministic algorithms; bit-identical reproduction was not confirmed, though each pair is validated to share an identical checkpoint, stream, and horizon. (viii) The severities were calibrated on a clean checkpoint from a separate training run; the calibration digest differs from the episode checkpoints, and severities are not re-validated per seed. (ix) The reported contrasts depend on $\tau = 0.5$ and the calibrated severities; we report descriptive group means over 12 groups without multiple-comparison correction, and the ordinal robustness to $\tau \in \{0.40, 0.55\}$ (Section VI-B) does not extend to its magnitude. (x) Compared with a first-alarm trigger (quorum 0.10, one-tick window), the quorum-0.30 arm is slower and incurs more downtime, the cost concentrating in Gaussian noise; the false-positive protection the quorum buys could not be quantified (no drift-free runs at the low thresholds), so our claims are scoped to its viability, zero false-positive rate on drift-free streams, and scenario-dependent advantage; scheduled, oracle, and centralized-detector comparators remain future work; since drift onset coincides with production start in every episode, calendar-scheduled retraining would act as an oracle rather than a fair baseline. (xi) Drift-free control episodes (15) produced zero false triggers (0/15), so the detector's false-positive rate under continuous monitoring is 0% within this setup.

VIII. Conclusion and Future Work

We studied reactive retraining in synchronous federated learning across the abrupt-to-gradual drift continuum, evaluating our reactive policy against a paired no-

retraining baseline over 60 episodes with service downtime as the primary metric. Detection succeeded in 59 of 60 episodes, the delay growing with the ramp duration for the visual corruptions; the reactive arm reduced the mean downtime from 408–562 s to 0–178 s (zero in the fog 200 s-ramp case); and long ramps preserved clean accuracy for the visual corruptions. Against the first-alarm comparator, the quorum-0.30 arm trades about 13.8 s of mean detection latency and about 20 s of mean downtime for corroboration of a collective decision; the advantage is scenario-dependent—on Gaussian noise the first-alarm trigger is markedly more robust (77.3 s vs. 140.1 s), while the collective trigger fires in all 20 staggered episodes (70–81% of the baseline downtime avoided) with a 0% false-positive rate over 15 drift-free controls. Future work includes: centralized-detector, oracle, and schedule-aware (randomized-onset) baselines to isolate the contribution of the detection mechanism; more seeds and a formal dispersion analysis; recurrent drift; non-IID partitions and heterogeneous client speeds; incremental drift; larger models; sensitivity of the detector's T, δ, and calibration bounds; the normalized detection delay (NDT) of the benchmark [3]; and publishing the protocol as a reusable benchmark for FL-under-drift evaluation [19].

Availability

The implementation and all artifacts—the 60 paired episodes, comparator arms, aggregated summaries, and per-seed tables—are available at [https://github.com/julio-amaral-oliveira/FederatedLearningSimulation].

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. AISTATS, PMLR 54, pp. 1273–1282, 2017.

[2] F. E. Casado, D. Lema, M. F. Criado, R. Iglesias, C. V. Regueiro, and S. Barro, "Concept drift detection and adaptation for federated and continual learning," Multimedia Tools and Applications, vol. 81, pp. 3397–3419, 2022.

[3] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, "A framework for evaluating and benchmarking concept drift detection methods," in Proc. KDD, 2026, doi: 10.1145/3770855.3819070.

[4] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, and P. B. Gibbons, "Federated learning under distributed concept drift," in Proc. AISTATS, PMLR 206, pp. 5834–5853, 2023.

[5] T. Chow, U. Raza, I. Mavromatis, and A. Khan, "FLARE: Detection and mitigation of concept drift for federated learning based IoT deployments," in Proc. IEEE IWCMC, pp. 989–995, 2023.

[6] I. Mavromatis, S. De Feo, and A. Khan, "FLAME: Adaptive and reactive concept drift mitigation for federated learning deployments," arXiv:2410.01386, 2024.

[7] A. Patel, D. S. Tomar, R. K. Pateriya, and R. Haripriya, "FL-MalDrift: A federated learning framework for malware detection under local concept drift," Scientific Reports, vol. 16, Art. 1821, 2026.

[8] Y. Han, D. Wu, and B. Varghese, "DriftGuard: Mitigating asynchronous data drift in federated learning," arXiv:2603.18872, 2026.

[9] P. Fu and M. Tang, "FedDAA: Dynamic client clustering for concept drift adaptation in federated learning," arXiv:2506.21054, 2025.

[10] Q. Zhou et al., "FedPLC: Federated learning with dynamic cluster adaptation for concept drift on non-IID data," Sensors, vol. 26, no. 1, Art. 283, 2026.

[11] L. Baier, T. Schlör, J. Schöffer, and N. Kühl, "Detecting concept drift with neural network model uncertainty," in Proc. 56th Hawaii Int. Conf. on System Sciences (HICSS), pp. 835–844, 2023.

[12] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in Proc. ICML, PMLR 48, pp. 1050–1059, 2016.

[13] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in Proc. SIAM Int. Conf. on Data Mining, pp. 443–448, 2007.

[14] J. L. Lobo, I. Laña, E. Osaba, and J. Del Ser, "On the connection between concept drift and uncertainty in industrial artificial intelligence," in Proc. IEEE CAI, pp. 171–172, 2023.

[15] Y. Ovadia et al., "Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift," in Proc. NeurIPS, pp. 13991–14002, 2019.

[16] V. Nguyen, C. Shui, V. Giri, S. Arya, A. Verma, F. Razak, and R. G. Krishnan, "Reliably detecting model failures in deployment without labels (D3M)," in Proc. NeurIPS, 2025.

[17] O. Solozobov, "Label-free detection of governance evidence degradation in risk decision systems," arXiv:2604.17836, 2026.

[18] L. Poenaru-Olaru, L. Cruz, A. van Deursen, and J. S. Rellermeyer, "Are concept drift detectors reliable alarming systems?" arXiv:2211.13098, 2022.

[19] O. A. Mahdi, E. Pardede, S. Bevinakoppa, and N. Ali, "Federated learning under concept drift: A systematic survey of foundations, innovations, and future research directions," Electronics, vol. 14, no. 22, Art. 4480, 2025.

[20] M. Böther et al., "Modyn: Data-centric machine learning pipeline orchestration," Proc. ACM on Management of Data, vol. 3, no. 1, Art. 55, 2025.

[21] M. F. Criado, F. E. Casado, R. Iglesias, C. V. Regueiro, and S. Barro, "Non-IID data and continual learning processes in federated learning: A long road ahead," Information Fusion, vol. 88, pp. 263–280, 2022.

[22] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in Proc. ICLR, 2019.

[23] K. Panchal et al., "Flash: Concept drift adaptation in federated learning," in Proc. ICML, PMLR 202, pp. 26931–26962, 2023.

[24] X. Chen et al., "Virtual concept drift detection and adaptation in federated data stream learning," Int. J. Data Science and Analytics, vol. 21, Art. 46, 2026.

[25] M. Stallmann, A. Wilbik, and G. Weiß, "Towards unsupervised sudden data drift detection in federated learning with fuzzy clustering," in Proc. IEEE FUZZ-IEEE, 2024.

[26] R. A. Bhope, K. R. Jayaram, P. Venkateswaran, and N. Venkatasubramanian, "Shift happens: Mixture of experts based continual adaptation in federated learning," in Proc. ACM Middleware, pp. 455–468, 2025.

[27] G. Canonaco, A. Bergamasco, A. Mongelluzzo, and M. Roveri, "Adaptive federated learning in presence of concept drift," in Proc. IEEE Int. Joint Conf. on Neural Networks (IJCNN), pp. 1–7, 2021.

[28] P. Lu, J. Lu, A. Liu, and G. Zhang, "Early concept drift detection via prediction uncertainty," in Proc. AAAI Conf. on Artificial Intelligence, vol. 39, no. 18, pp. 19124–19132, 2025.

[29] B. Zhang, H. Guan, H. K. Lee, R. Liu, J. Zou, and L. Xiong, "FedSGT: Exact federated unlearning via sequential group-based training," arXiv:2511.23393, 2025.